# MATE (Multi-Agent Tree Engine) System

Comprehensive System Architecture and Design Specification

Google ADK Team and MATE Architecture Board

July 2026

## Contents

**1 Executive Summary** **2**

**2 System Architecture & Dual-Server Topology** **2**
2.1 Auth Server (`auth_server.py`) . . . 3
2.2 ADK Server (`adk_main.py`) . . . 3

**3 Agent Management & Execution Layer** **3**
3.1 Agent Manager (`shared/utils/agent_manager.py`) . . . 3
3.2 Agent Orchestration Types . . . 3
3.3 Multi-Model & LiteLLM Integration . . . 3

**4 Workflow Agents & Graph Runtime** **3**
4.1 Graph Specification & Edges . . . 4
4.2 Graph Resilience & Retries . . . 4
4.3 Human-in-the-Loop Approvals . . . 4
4.4 App-Wide Plugin Mode . . . 4

**5 Tool Integration & Factory Layer** **4**
5.1 Supported Tool Types . . . 4
5.2 Agent Self-Modification (`create_agent_tool`) . . . 4

**6 Database Architecture & Migration Engine** **5**
6.1 Core ORM Tables . . . 5
6.2 Automated Migration Engine . . . 5

**7 Security & Identity Governance** **5**
7.1 Authentication Framework . . . 5
7.2 Single Sign-On (SSO) OAuth 2.0 . . . 6
7.3 Role-Based Access Control (RBAC) . . . 6

**8 Autonomous Trigger Engine** **6**
8.1 Trigger Types . . . 6
8.2 Output Routing & Destinations . . . 6

**9 OpenTelemetry Distributed Tracing** **6**

**10 Extensibility & Integration Bridges** **7**
10.1 Embeddable Chat Widget . . . 7
10.2 OpenAI Compatibility Bridge . . . 7
10.3 Slack Bot Bridge . . . 7

**11 Dynamic Memory & Bootstrap Protocol** **7**

**12 Deployment Architecture & Production Hardening** **8**
12.1 Hardened Service Abstractions . . . . . 8
12.2 Rate Limiting & Budget Controls . . . . . 8
12.3 Standalone Building (`build_standalone_agent.py`) . . . . . 8

# 1 Executive Summary

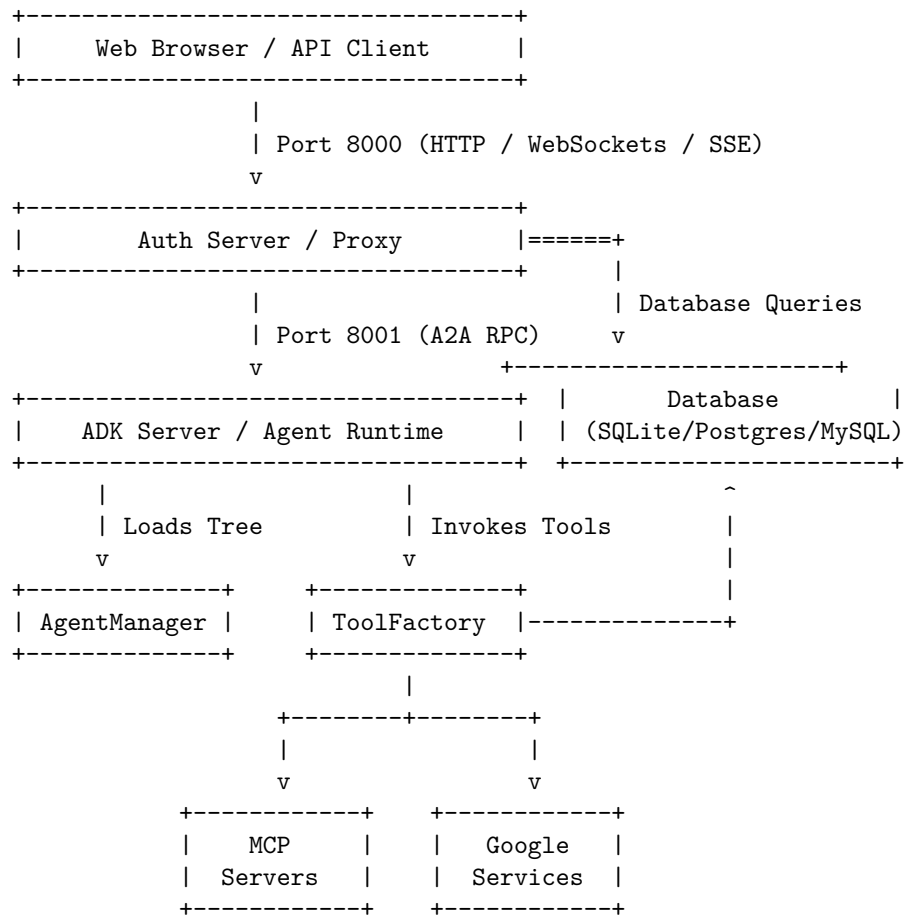
**MATE (Multi-Agent Tree Engine)** is a production-grade enterprise orchestration platform built on top of **Google's Agent Development Kit (ADK) 2.x**. It extends raw ADK capabilities by adding database-driven agent lifecycle management, multi-model LLM abstraction, Role-Based Access Control (RBAC), Model Context Protocol (MCP) server/client integration, distributed tracing, automated scheduling, and a user-friendly management dashboard.

MATE bridges the gap between raw, static code-defined agents and dynamic, multi-tenant enterprise applications. It allows developers to configure complex, hierarchical agent trees and graphs, assign granular security profiles, monitor token costs in real-time, and run autonomous workflows without code redeployments.

---

# 2 System Architecture & Dual-Server Topology

MATE utilizes a **dual-server architecture** designed to isolate client-facing security, rate limiting, and dashboard delivery from the internal agent execution runtime.



### 2.1 Auth Server (`auth_server.py`)

Running on port **8000**, the Auth Server serves as the entry point for all external traffic. It is written in FastAPI and handles: - **Authentication**: HTTP Basic Auth, Bearer Token Auth, and OIDC/OAuth 2.0 Single Sign-On (SSO). - **Dashboard UI**: Renders the management portal and admin console. - **Developer API**: Hosts `/dashboard/api/` endpoints for agent management, user CRUD, token logging, and system actions. - **OpenAI Bridge**: Hosts the OpenAI-compatible `/v1` endpoints, translating incoming completions queries to internal agent execution runs. - **WebSocket Proxying**: Streams real-time agent output tokens directly to clients. - **Reverse Proxy**: Filters and forwards authorized runtime calls to the ADK Server.

### 2.2 ADK Server (`adk_main.py`)

Running on port **8001**, the ADK Server is the core agent runtime. It is locked down and does not accept direct public connections. It handles: - **Agent Lifecycle**: Compiles and executes agent runtimes inside isolated execution contexts. - **System Services**: Registers and administers ADK framework services (e.g., Artifact Service, DBCredentialService, DBMemoryService, and DatabaseSessionService). - **Agent-to-Agent (A2A) Protocol**: Executes messaging between agents inside the tree.

---

## 3 Agent Management & Execution Layer

### 3.1 Agent Manager (`shared/utils/agent_manager.py`)

The `AgentManager` is responsible for building the unified agent tree. At server startup, it executes a merge strategy: 1. **Hardcoded Agents**: Scans the `agents/` folder and loads class definitions. 2. **Database-Driven Agents**: Queries the `agents_config` table. 3. **Hierarchy Stitching**: Recursively traverses `parent` and `sub-agent` relationships, creating parent-child pointers to form a tree structure. 4. **Caching**: Caches initialized agents in memory to avoid repetitive DB hits.

### 3.2 Agent Orchestration Types

MATE supports five primary patterns of agent execution: - `llm`: A single agent node bound to a model with specific system instructions and tools. - `sequential`: Invokes configured sub-agents in a sequential chain, passing context down the line. - `parallel`: Runs configured sub-agents concurrently, aggregating their responses. - `loop`: Runs a sub-agent repeatedly until a completion criteria is met or max iterations is reached. - `graph`: Workflows governed by the ADK 2.x Graph Runtime.

### 3.3 Multi-Model & LiteLLM Integration

The Model Factory (`shared/utils/utils.py::create_model`) provides provider-agnostic model creation: - **Direct Gemini Support**: For high-throughput Google models, it utilizes the native `google-genai` SDK using the `GOOGLE_API_KEY` environment variable. - **LiteLLM / OpenRouter**: For multi-provider flexibility, it uses the LiteLLM SDK. Models prefixed with `openrouter/` (e.g., `openrouter/anthropic/claude-3.5-sonnet`) automatically map to OpenRouter.

---

## 4 Workflow Agents & Graph Runtime

MATE fully integrates the **ADK 2.x Graph Runtime** for compiling non-linear workflows configured via the dashboard.

### 4.1 Graph Specification & Edges

Graph layouts are declared in JSON inside the agent's `planner_config` field: - **Edges**: Paths connecting execution nodes. Sequential edges are declared as simple arrays (e.g., `["START", "agent_1"]`). Conditional routes use dictionary entries mapping target routes: `json { "from": "intent_router", "to": "refund_agent", "route": "refund" }` - **Router Nodes**: Deterministic nodes that evaluate agent outputs, checking `state[state_key]` against exact/substring routes to determine the next node to trigger. - **Join Nodes**: Auto-detected nodes (by matching `"join"` in their name) that act as synchronization points for concurrent fan-in edges.

### 4.2 Graph Resilience & Retries

MATE implements framework-level retries for nodes in the graph: - **`retry_config`**: Standard backoff parameters for the graph agent itself. - **`node_retry`**: Fine-grained retry limits per-node. - *Best Practice*: Let exceptions bubble up from tools; trapping exceptions with overly broad `try-except` blocks hides failures and blocks the framework retry loops.

### 4.3 Human-in-the-Loop Approvals

Tools can be flagged for human confirmation. When `require_confirmation` is set: 1. The tool execution pauses and registers a pending state. 2. The user is prompted on the dashboard to **Approve** or **Reject** the call. 3. If `RESUMABILITY_ENABLED=true` is set, the execution state is saved to the DB, allowing full recovery and resumption from the paused node.

### 4.4 App-Wide Plugin Mode

When `MATE_PLUGINS_ENABLED=true` is configured, callbacks (RBAC, guardrails, tracing, cost logs) are registered once at the application level rather than per-agent. This covers all agents running in the process, including those spawned dynamically via the `create_agent_tool`.

---

## 5 Tool Integration & Factory Layer

The `ToolFactory` (`shared/utils/tools/tool_factory.py`) instantiates and binds tools to agents based on the agent's `tool_config` and system configurations.

### 5.1 Supported Tool Types

- **MCP Tools**: Connects standard Model Context Protocol servers over stdio (defined in `mcp_servers_config`).
- **Google Services**: Native Google Drive (file operations) and Google Search tools.
- **CV Analysis Tools**: Computer vision utilities for image inspection.
- **Image Generation Tools**: Connects DALL-E or local generative setups.
- **File Search (Vector Stores)**: Performs semantic document searches.
- **User Profile Tools**: Injects logged-in user profile details to personalize prompts.
- **Custom Function Tools**: Python functions defined in `shared/utils/tools/custom_tools.py`.
- **Dynamic Memory Block Tools**: Performs CRUD actions on memory blocks.

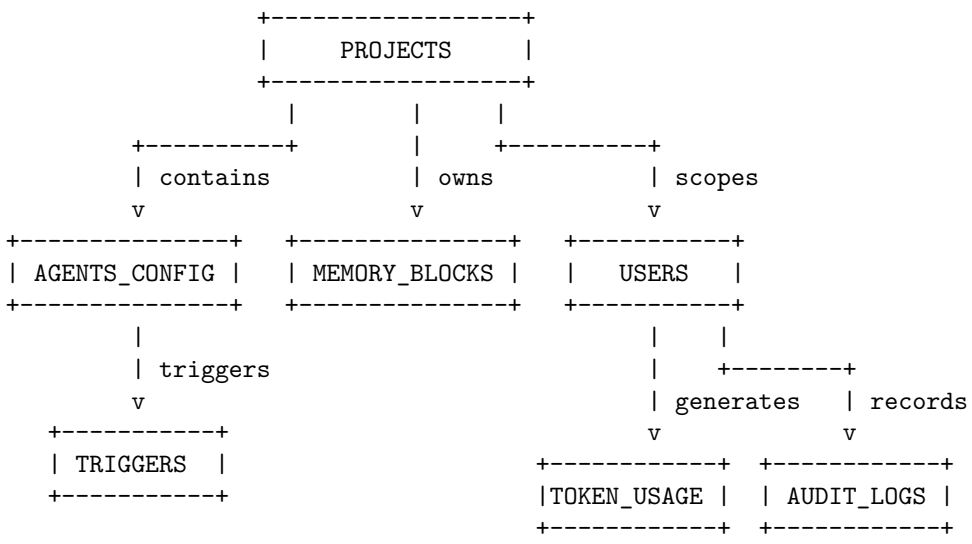
### 5.2 Agent Self-Modification (`create_agent_tool`)

Allows trusted agents to dynamically register, update, or delete other agents in the database at runtime. This creates self-improving and self-orchestrating agent loops.

---

# 6 Database Architecture & Migration Engine

MATE includes a production-grade database abstraction layer using SQLAlchemy ORM.




## 6.1 Core ORM Tables

- `projects`: Handles multi-tenant logical partitioning of all agents and data.
- `agents_config`: Stores agent parameters, parent pointers, system instructions, model names, planners, and JSON-encoded tool/planner configurations.
- `users`: Manages users, credentials, and roles.
- `token_usage_logs`: Logs request/response tokens, model names, and execution costs.
- `guardrail_logs`: Tracks policy and guardrail violations.
- `audit_logs`: General security audit trails.
- `memory_blocks`: Stores persistent dynamic instructions and facts.
- `widget_api_keys`: Manages access tokens for embeddable chat widgets.

## 6.2 Automated Migration Engine

Database schema updates are managed through a custom python migration manager (`shared/migrate.py`). - **Path structure**: Separate folders for database dialects: `shared/sql/migrations/{sqlite, postgresql, mysql}/`. - **Startup Sync**: On system boot, `shared/migrate.py run` executes sequentially. It cross-references the local migrations table, checks checksum integrity, and auto-applies pending SQL scripts before spinning up servers.

---

# 7 Security & Identity Governance

## 7.1 Authentication Framework

MATE enforces security through FastAPI dependencies: - **HTTP Basic Auth**: Default username and password defined via environment variables. - **Bearer Tokens**: JWT sessions for web clients. - **Personal Access Tokens (PATs)**: SHA-256 hashed keys generated by users (`mate_pat_...`) for secure programmatic access (e.g. from IDE extensions).

### 7.2 Single Sign-On (SSO) OAuth 2.0

MATE supports **Google OIDC** and **GitHub OAuth 2.0** login flows powered by `Authlib`: - Enforces **Authorization Code Flow with PKCE** for secure, interception-resistant authorization. - Basic Auth and SSO providers can run concurrently, providing fallback paths for local scripts.

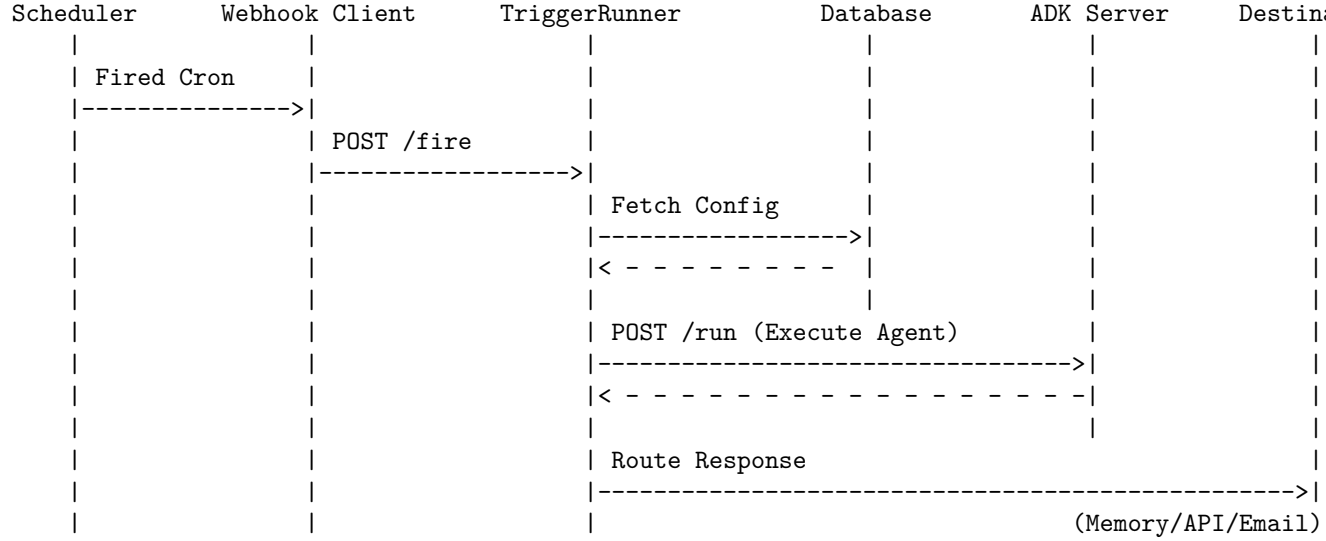
### 7.3 Role-Based Access Control (RBAC)

User permissions are verified via callbacks (`rbac_callback.py`): - Roles are defined as JSON arrays in the `users` table. - Agents specify their permissions via the `allowed_for_roles` array. - During tree execution, MATE validates that the executing user's role set overlaps with the agent's allowed role set.

---

## 8 Autonomous Trigger Engine

The Trigger Engine enables **autonomous agent runs** that execute without human prompt initiation.




### 8.1 Trigger Types

- `cron`: Fires based on standard 5-field UTC cron expressions. Backed by `APScheduler` configured with `coalesce=True` to prevent catch-up bursts.
- `webhook`: External systems POST to `/triggers/{id}/fire`. Authenticated via SHA-256 hashed **Fire Keys** passed in headers or query parameters.

### 8.2 Output Routing & Destinations

1. `memory_block`: Saves execution outputs directly into a project-scoped memory block.
2. `http_callback`: POSTs JSON payloads to a custom URL with configurable headers and timeouts.
3. `email`: Sends text summaries to configured recipients via SMTP.

---

## 9 OpenTelemetry Distributed Tracing

MATE integrates OpenTelemetry to record execution spans across all agent tasks, LLM prompts, tool executions, and database queries.

- **Jaeger / Grafana Tempo / Datadog**: Exports spans via OTLP collector protocol when `OTEL_TRACING_ENABLED=true`.
- **Database Span Storage**: Saves spans directly to the database. The MATE Dashboard reads these logs to render a visual execution waterfall trace viewer.
- **W3C Trace Context**: Outgoing HTTP tool requests propagate `traceparent` headers to maintain end-to-end tracing across external systems.

---

# 10 Extensibility & Integration Bridges

## 10.1 Embeddable Chat Widget

MATE exposes an embeddable widget (`widget.js`) enabling developers to embed chat panels on external websites with a single `<script>` block. - Authenticated via **Widget API Keys** scoped to a project. - Exposes a JS API (`window.MateChat.open()`, `close()`, `toggle()`) for custom triggers.

## 10.2 OpenAI Compatibility Bridge

Exposes `/v1/models` and `/v1/chat/completions` endpoints. - External developer tools (e.g. VS Code Continue, Cline/Roo Code, OpenCode) can consume MATE agents as standard LLMs by setting their base URL to `http://localhost:8000/v1` and passing a PAT.

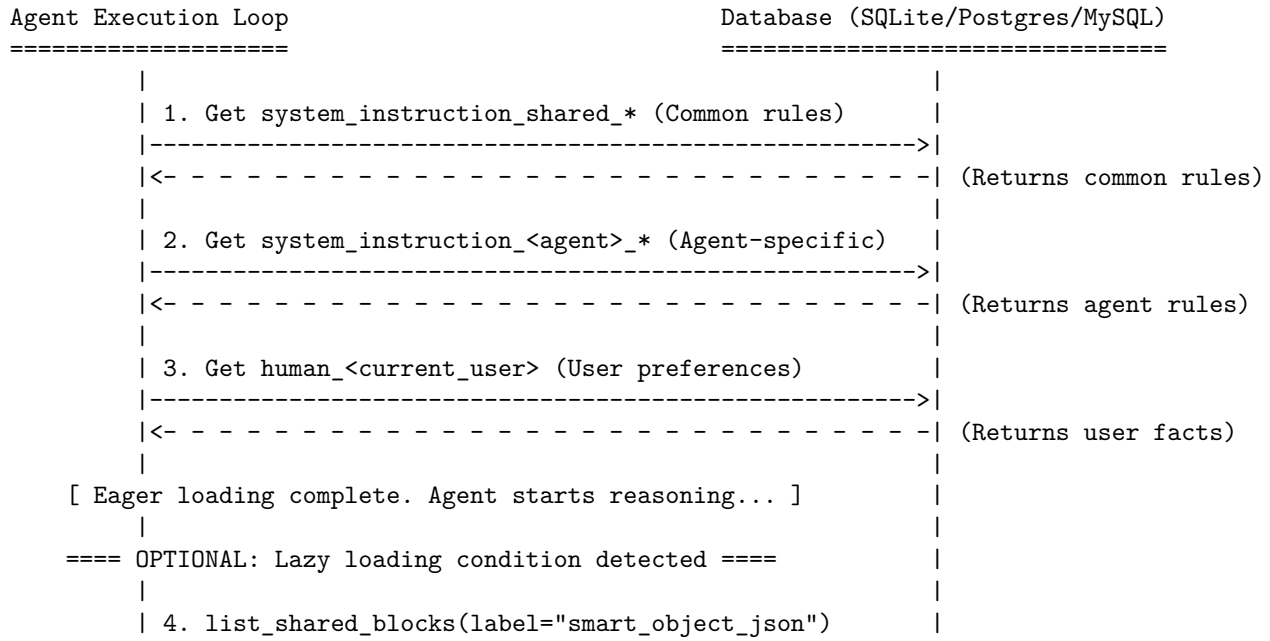
## 10.3 Slack Bot Bridge

Connects agents to Slack channels. Integrates event handlers to listen to channel mentions, parse threads, execute root agents, and post answers back.

---

# 11 Dynamic Memory & Bootstrap Protocol

MATE utilizes the **Bootstrap Pattern** to decouple system instructions from hardcoded code or static database records.



```
|-------------------------------------------------------->|
|<- - - - - - - - - - - - - - - - - - - - - - - - - - - - -| (Returns JSON Schema)
|                                                         |
```

- **Eager Loading**: At execution boot, the agent queries the database to load all shared system instruction blocks and user personalization blocks (`human_<username>`).
- **Lazy Loading**: Avoids bloating context windows by loading large schemas (e.g., frontend visualizer JSON structures) only when specific user query triggers are detected.

---

# 12 Deployment Architecture & Production Hardening

## 12.1 Hardened Service Abstractions

MATE replaces standard local filesystems with production-ready services: - **Artifact Service**: Supports S3, Supabase storage, or local directories. - **Session Service**: Keeps state persisted across backend nodes in the database. - **Credential Service**: Vaults API keys securely.

## 12.2 Rate Limiting & Budget Controls

If `RATE_LIMIT_ENABLED=true` is set, MATE monitors token expenditure. It checks user and project usage metrics before agent execution. If the user's budget is exceeded, it gracefully halts execution with a budget error.

## 12.3 Standalone Building (`build_standalone_agent.py`)

MATE includes a script that compiles an entire agent project config, SQLite database, and execution engine into a single self-contained binary using PyInstaller, facilitating easy distribution and local execution.